

Angry Agents

A Multi-Agent Persona Simulation and Evaluation Platform

AI Design 2026 — Final Project Report

May 2026

Team:

Alfonso Antognozzi Davide Cabitza Gabriele Fronzoni
Kevin Shimaj Marco Massa



Contents

1	Introduction	2
1.1	Interaction modes	2
1.2	Persona types	2
1.3	User roles	2
2	Project Evolution	2
2.1	Phase I — Foundation (May 5–13)	2
2.2	Phase II — Core Pipeline (May 14–24)	3
2.3	Phase III — Integration (May 25–27)	5
2.4	Phase IV — Polish and Hardening (May 28–31)	6
3	System Architecture	6
3.1	Technology stack	7
3.2	Layered architecture	7
3.3	Database schema	7
4	Persona Extraction and Profiles	8
4.1	Source material	8
4.2	Two-phase extraction pipeline	8
4.3	Structured profile schema	8
5	Context Management	9
5.1	Token budget per turn	9
5.2	Mitigation mechanisms	9
6	Chat Simulation	10
6.1	PersonaAgent	10
6.2	TurnScheduler	10
6.3	GroupChatSession	10
6.4	DM chat	10
7	Evaluation Framework	10
7.1	Four fidelity dimensions	11
7.2	Phase 1 — Independent evaluation	11
7.3	Statistical metrics	11
7.4	Phase 2 — Deliberation (design)	12
8	MCP Layer	12
8.1	Architecture	12
8.2	Tool tiers	12
9	Knowledge Graph	13
10	State-of-the-Art Positioning	13
11	Open Questions and Future Work	14
12	References	14

Introduction

Angry Agents is a web application in which users interact with AI persona-agents that faithfully represent real-world or fictional characters — politicians, influencers, movie and tv series characters, and more. Conversations can be evaluated by 20 judge-agents across four fidelity dimensions: persona identification, individual fidelity, group fidelity, and behavioral fidelity.

The project addresses a research question at the intersection of multi-agent systems, LLM-based role-playing, and automated evaluation:

Can an LLM, conditioned on a structured persona profile extracted from source material, faithfully simulate a real or fictional character in a multi-turn conversation, to the point that independent judge-agents can correctly identify it?

Interaction modes

The platform supports two conversation types:

DM One user converses one-on-one with a single persona-agent. The agent responds to the user's messages.

Group chat The user sets a topic and selects 2–8 agents. The agents start to converse autonomously and the user can observe and inject messages.

Persona types

Fiction Characters extracted from movie scripts or TV screenplays (e.g. Batman, Darth Vader, Bart Simpson).

Real-world People extracted from podcast transcripts, YouTube videos, and social-media corpora (e.g. Barack Obama, Andrej Karpathy, Bernie Sanders).

User roles

Common user Chat with agents, manage own conversations.

Admin Manage personas, launch judge evaluations, view metrics.

Project Evolution

The development did not follow a waterfall plan but grew organically: each week surfaced new problems that reshaped the design of the previous week. This section traces that evolution, describing what was built, what broke, and what had to be rebuilt.

Phase I — Foundation (May 5–13)

Starting from the question. Our work started from asking ourselves *"What's the real application scope?"*. We started defying the problem and understand what the application should have done. The main challenge was to understand how to conduct the evaluation,

Phase	Period	Main output
I - Foundation	May 5–13	Problem definition, SOTA, first DB schema, first personas, first UI
II - Core pipeline	May 14–24	Full REST API, auth, judge classes, persona runtime, chat engine
III - Integration	May 25–27	RAG pipeline, Hungarian fix, two-phase extraction, UI wired, MCP
IV - Polish	May 28–31	UI polish, evaluation refinements, batch metrics, MCP hardening

Figure 1: Project phases and their primary deliverables.

since the definition of clear eval parameters and pipeline was crucial to realize a good judgment system.

To help us to better understand problem scope, we searched and analyzed some papers. Park et al. [2023] showed that long-running agents drift toward generic agreement without memory and reflection mechanisms. Zhou et al. [2025] demonstrated that *personal detail features* — not surface vocabulary — are the strongest predictor of behavioral fidelity. Jiang et al. [2024] proved that raters who know they are reading AI output systematically underestimate personality fidelity, directly motivating a blind evaluation protocol.

First data model. The earliest database design had six tables: `Agents`, `Chat`, `Chat_messages`, `Judges`, `Judge_evaluation`, and two separate context tables for fiction and podcast personas. At this stage, the most we made an important design decision: the `author` field in `Chat_messages` would store a cryptographic hash of the agent’s identity rather than a foreign key back to `Agents`. This single choice encoded the core requirement - judges must not be able to link a message to a known persona - at the schema level, making it impossible to violate accidentally.

First personas. Three scraping pipelines were started in parallel: YouTube transcript extraction for real-world personas (politicians, podcasters), movie-script PDF parsing for fictional characters, and IMSDB HTML scraping for additional screenplays. The first eleven fictional profiles were extracted: Darth Vader, Yoda, Indiana Jones, Jack Sparrow, Luke Skywalker, Batman, Joker, and others. Each profile was a structured JSON document capturing speaking style, worldview, emotional tells, and signature phrases — a format informed directly by the Zhou et al. finding on personal detail importance.

First UI. A React prototype with ten components covered the full user journey: authentication, persona library, DM chat wizard, group chat wizard, active chat, and admin dashboard. It ran entirely on static mock data. The purpose was not to ship a product but to expose interaction design questions early, before the backend was complete.

Phase II — Core Pipeline (May 14–24)

Rethinking the schema. The two separate context tables from Phase I were a premature distinction. As the personas grew in complexity, both fiction and real-world agents needed the same structured JSON profile; the only difference was the `source_type`

field. The schema was consolidated: one `Agent_context` table with a `signature_phrases` JSON column. A `Topic` table was added to give group chats a shared theme. All entities gained `created_at` / `updated_at` / `deleted_at` timestamps enforced by database triggers, enabling soft-delete across the board.

The most significant structural addition was the `User` table. The first schema had no concept of platform users at all — agents, chats, and evaluations existed in isolation. Adding users required propagating a `Created_by` foreign key through four tables (`Group_chat`, `Topic`, `Agents`, `Chat_messages`) and introducing role-based access (`common` vs. `admin`). This also unlocked proper authentication: JWT access tokens (15-minute expiry) paired with `httpOnly` refresh tokens whose SHA-256 hash is stored in a dedicated `RefreshToken` table.

Persistence and API layer. The complete backend was built in this phase: raw-SQL CRUD repositories for all eight entities, a service layer enforcing business invariants (the 20-evaluation cap per chat, HMAC author token generation, PBKDF2-SHA256 password hashing with 260 000 iterations), and a FastAPI application with nine route files and Pydantic schemas. A test suite of 291 tests covered all repositories and services, including boundary cases for soft-delete cascades, composite primary keys, password-hash opacity, JWT token lifecycle, and slug collision handling.

Judge architecture. The evaluation design required judges with different focuses. Three concrete judge classes were implemented: **StyleJudge** (vocabulary, tone, rhetorical habits), **IdeologyJudge** (political views, moral stances), and **GeneralJudge** (all traits combined). Each extended an abstract `BaseJudge` with four evaluation methods — persona identification, individual fidelity, group fidelity, and behavioral fidelity. Prompts were extracted from Python strings into Jinja2 `.j2` templates, making them versionable and enforceable (every template required explicit motivation-before-rating, a 1–5 rubric, and JSON output).

Persona runtime and group chat engine. The `PersonaAgent` class wrapped a database agent row into a live, chat-ready object. Rather than treating all agents as identical, the class derived behavioral parameters from the persona’s own profile: `dominance_weight` from social-positioning keywords, `cooldown_turns` from the rhythm field in `core_style`, and `burst_size` from sentence-shape descriptors. A high-dominance agent would speak more and recover faster; a reserved agent would yield turns naturally.

The `GroupChatSession` orchestrator connected a `TurnScheduler` (weighted-random or round-robin) with a `ContextWindow` (rolling or selective history trimming) to run fully autonomous multi-agent conversations. A dedicated `DMSession` handled the simpler one-on-one path. Both modes used the same persona engine but different prompt templates.

First scalability wall. Running persona identification with three judges against a 40-message transcript and seven personas already exceeded the local 120-second timeout. With the full design calling for twenty judges and a database of 100+ personas, the evaluation pipeline was clearly unviable at this scale. This problem was deferred to Phase III.

Phase III — Integration (May 25–27)

Phase III was the most intensive period of the project. All major subsystems converged simultaneously, and several assumptions from earlier phases had to be corrected under pressure.

The scalability problem, solved. The root cause of the evaluation bottleneck was that each judge had to consider every persona in the database for every message. The fix was a Retrieval-Augmented Generation (RAG) pre-filtering layer. Each persona’s structured profile was split into typed semantic chunks (`style`, `voice`, `worldview`, `behavior`, `quote`), embedded with `text-embedding-3-small`, and stored in a ChromaDB index. Before scoring, each judge queried the index to receive a shortlist of the 20 most likely candidates, ranked by field-weighted cosine similarity and re-ranked by Maximal Marginal Relevance (to prevent a single “attractor” persona from dominating the shortlist). This reduced LLM calls from $\sim 2\,000$ to ~ 80 per run — a 97% reduction — and brought latency from 33 minutes to ~ 30 seconds.

A flaw in persona identification. An independent issue surfaced in the evaluation logic: the greedy argmax that assigned each persona to its highest-scoring author could assign the same author to multiple personas simultaneously. In a chat with eight agents, a single highly distinctive author could “win” several persona slots, inflating accuracy artificially. The fix was to replace argmax with the **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`), which finds the assignment that maximises total score subject to the constraint that each author maps to exactly one persona and vice versa. This change made persona identification results valid.

Deeper persona extraction. Source transcripts alone could not fully specify some persona traits — specifically, catchphrases and syntactic tics that are widely known but rarely appear verbatim in any single transcript. The extraction pipeline was redesigned as a two-stage process. The first call used `gpt-4o-mini` with the full transcript to ground all behavioral fields in real observed speech. The second call used `gpt-4o` with *no transcript* to extract pure model-knowledge fields (`avored_words`, `structural_patterns`, `do_not_say`) from the model’s parametric knowledge. Each extracted entry had to fail a “similar figure” discriminability test: if the same line could plausibly come from a confusable persona, it was discarded. The total extraction cost for 80 personas was **\$0.57** (1.19M tokens).

Closing the UI gap. The React prototype had been running on static mock data since Phase I. In this phase, all screens were wired to live API calls. A shared `api.js` HTTP client and an `agents-store.jsx` React context replaced `data.js`. The admin dashboard was connected to the evaluation output, rendering persona identification accuracy, the confusion matrix, per-persona fidelity scores, and the turn-distribution Gini coefficient in real time.

MCP integration. To allow AI agents to interact with the platform programmatically, a Model Context Protocol server was implemented. It exposed two tiers of tools: read-only operations requiring no confirmation, and write operations requiring an explicit preview-

then-confirm step from the user. The LLM cannot stop a chat, launch a judging run, or post a message without the user approving the action first.

Statistical evaluation engine. The complete `src/eval/` package was finished: four metric modules covering persona identification (binomial exact CI, Cohen’s kappa, confusion matrix), individual fidelity (median, IQR, bootstrap 95% CI), group fidelity (Gini coefficient, cosine distance matrix), and deliberation convergence (variance reduction, F-test). All non-parametric estimates share a single `bootstrap_ci()` helper with 10 000 resamples.

Phase IV — Polish and Hardening (May 28–31)

The final phase had no single large feature. Instead, it addressed a collection of gaps and inconsistencies that only became visible once the full system was running end-to-end.

Evaluation refinements. The admin dashboard gained Mean Absolute Deviation (MAD) between all judge-type pairs, making systematic disagreement between style, ideology, general, and behavioral judges visible at a glance. A batch evaluation pipeline was added to aggregate results across multiple chat sessions: for fewer than 20 sessions it uses bootstrap resampling; for 20 or more it switches to the Central Limit Theorem normal approximation.

UI and UX corrections. Real-world personas had been displaying their surname as the subtitle in the agent library — a fallback that exposed an internal implementation detail. The fix was to read the `occupation` field from the persona’s structured profile and use it as the subtitle instead, making the library read as “AI researcher” rather than “Karpathy”. Other fixes: a pause button for group chats, filters on the chat list, responsive layout adjustments, and a light theme.

MCP hardening. The MCP read and write tool schemas were revised to match the current API. The documentation file was updated to reflect all available tools, their parameters, and the confirmation protocol for write operations.

Knowledge graph. A static knowledge graph of the entire codebase was generated and committed: 6 920 nodes and 8 337 edges across 20 community clusters, providing a queryable map of architectural relationships between components.

System Architecture

Technology stack

Layer	Technology
Backend API	Python 3.12, FastAPI, Uvicorn
Database	SQLite (raw SQL, WAL mode, FKs enforced)
Schema/validation	Pydantic v2
Auth	JWT (HS256), PBKDF2-SHA256, httpOnly cookies
LLM backend	OpenAI (gpt-4o-mini) or Ollama (mistral)
RAG	ChromaDB, text-embedding-3-small
Frontend	Vanilla React/JSX, no bundler
Testing	pytest, in-memory SQLite fixtures
MCP	FastMCP (src/mcp/mcp_server.py)

Layered architecture

The backend follows a strict layered pattern:

1. **Routes** (src/API/routes/) — HTTP layer only. Receives requests, calls services, returns responses. Never touches SQL directly.
2. **Services** (src/db/services/) — Business logic. Calls repositories, enforces invariants (e.g. 20-evaluation cap, slug uniqueness).
3. **Repositories** (src/db/repositories/) — Raw SQL CRUD. Every entity exposes: `create`, `get`, `get_by_slug`, `update`, `delete(hard=False)`, `query(filters, limit, offset)`.
4. **Models** (src/db/models/) — Python dataclasses mirroring DB tables.

Key invariant: SQL lives only in `api/`. Agent and evaluation code receives plain Python objects.

Database schema

The final schema (v3, implemented in `src/db/models/base.py`) comprises ten tables:

Table	Purpose	Notable constraints
User	Platform users	Role: common admin; slug UNIQUE
Agents	Persona profiles	Summary: JSON profile; Created_by FK
Agent_context	Context chunks	Signature_phrases: JSON phrases
Topic	Chat themes	Title UNIQUE
Group_chat	Chat sessions	status: pending running done stopped error
Chat_agent	Chat↔Agent junction	Composite PK (id_chat, id_agent)
Chat_messages	Messages	author: name+surname+DIGEST; no FK to Agents
Judges	Judge instances	Role: style ideology general behavioral
Judge_evaluation	Evaluation scores	Composite PK; max 20 per chat enforced in service
RefreshToken	JWT refresh tokens	Stores SHA-256 hash, never plaintext

Soft-delete pattern. All tables have `deleted_at`. Queries always filter `WHERE deleted_at IS NULL`. Hard delete is admin-only. Slug uniqueness is `UNIQUE WHERE deleted_at IS NULL`.

Author anonymisation. `Chat_messages.author = HMAC-SHA256(ANGRY_AUTHOR_SECRET, name:surname)`. The hash is computed server-side by `ChatMessageService`. No foreign key from `Chat_messages` to `Agents` exists — judges can distinguish different message sources without knowing which agent sent what.

Persona Extraction and Profiles

Source material

- **Fiction personas:** movie scripts (IMSDB HTML, PDFs), TV screenplays. Challenge: sparse material — a character may have only a few dozen lines.
- **Real-world personas:** YouTube transcripts, podcast audio, Twitter/X threads. Challenge: extracting ideology and behavior from naturalistic speech rather than written dialogue.

Target source material: **10 000–50 000 tokens per persona** to capture not just surface vocabulary but deeper patterns: argumentation style, emotional triggers, social positioning.

Two-phase extraction pipeline

The extraction pipeline (`src/agents/personas/extract_profile.py`) was redesigned in Phase III to use two separate LLM calls:

Phase 1 — Transcript-grounded fields. Uses `gpt-4o-mini` with the full source transcript. Extracts behavioral fields that require grounding in real observed speech: `core_style`, `worldview`, `self_image_vs_reality`, `emotional_tells`, `situational_behavior`, `escalation_pattern`, `relationship_matrix`, `annotated_quotes`.

Phase 2 — Model-knowledge fields. Uses `gpt-4o` with *no transcript*. Extracts fields the model knows from parametric knowledge: `vocabulary_fingerprint.favored_words`, `speech_signature.structural_patterns`, `do_not_say`. Each entry must fail a “similar figure” discriminability test (i.e. it cannot be equally attributed to a confusable persona). Total cost for 80 personas: \$0.57 (1.19M tokens).

Structured profile schema

Each profile is a JSON document stored in `Agents.Summary`. Key fields:

Field	Content
<code>core_style</code>	Default register, sentence shape, rhythm, two registers
<code>humor</code>	Style, frequency, mechanism
<code>speech_signature</code>	Naming behavior, structural patterns
<code>vocabulary_fingerprint</code>	Favored words, avoided words, domain jargon, fillers
<code>worldview</code>	Key beliefs/values as key-value pairs
<code>self_image_vs_reality</code>	Self-image, reality, gap behavior
<code>emotional_tells</code>	When guarded, afraid, grieving, in control
<code>situational_behavior</code>	When outmatched, disagreeing, protecting, proven wrong
<code>escalation_pattern</code>	How the character escalates conflict

<code>conversation_goals</code>	List of what the character tries to achieve
<code>relationship_matrix</code>	Per-character relationship descriptions
<code>knowledge_domains</code>	Expert / surface / ignorant lists
<code>social_positioning</code>	Desired position, actual dynamic, contradiction
<code>annotated_quotes</code>	Quote + context + what it illustrates
<code>do_not_say</code>	Lines that would break character, with reason

Context Management

Long multi-turn conversations present three distinct problems for persona fidelity:

1. **Grounding** — the model has no inherent knowledge of the persona and must receive its profile on every inference call.
2. **Character drift** — after ~ 30 – 40 turns, the conversational context dominates the system prompt and the agent converges toward a generic LLM voice.
3. **Group dynamic drift** — in group chats with 8 agents, they converge toward similar tone and positions (excessive agreeableness, Park et al., 2023).

Token budget per turn

Component	Tokens (approx.)
Structured profile (cached)	$\sim 1\,000$
RAG chunks (dynamic)	~ 400
Compressed history (dynamic)	~ 300
Self-anchor / reflection	~ 200 (savings $\approx 92\%$)
Last 5 turns verbatim	~ 600
User message	~ 50
Total per turn	$\sim 2\,550$
<i>vs. raw transcript</i>	<i>$\sim 30\,000$</i>

Mitigation mechanisms

Sliding window + compression. Only the last $K = 5$ turns are passed verbatim. Every 10 turns, the older history is compressed into ~ 200 – 300 tokens via an LLM call and stored as a summary.

Reflection. Every $N = 15$ – 20 turns, the agent reviews its last messages against its profile and produces a 150–200-token self-anchor injected before the next reply.

Perturbation (group chat only). Every 10–15 turns per agent, a private signal is injected: “The conversation is moving toward consensus. React in a way that reflects your genuine position.” This fires on a per-agent basis and is not visible to other agents.

Drift detection. Pairwise cosine similarity between agents’ last-turn message embeddings is tracked. A rising similarity above threshold 0.85 triggers early reflection or perturbation.

Prompt caching. The structured profile is passed with a cache breakpoint after the system prompt (Anthropic ephemeral cache, 5-min TTL), saving 60–80% of input token cost on the static portion after the first turn.

Chat Simulation

PersonaAgent

`PersonaAgent` (`src/agents/personas/persona_agent.py`) is the core runtime class for a single persona in a chat:

- `bind_to_chat(chat_id, topic)` — resolves profile from DB, derives `dominance_weight` from `social_positioning` keywords, `cooldown_turns` from `core_style.rhythm`, `burst_size` from `core_style.sentence_shape`.
- `respond(history, turn_count)` — renders the Jinja2 prompt template, calls the LLM, returns the reply string.

TurnScheduler

Selects the next speaking agent:

- `round_robin` — cycles through agents in fixed order.
- `weighted_random` — samples proportional to `_effective_weight`, which penalises an agent within its `cooldown_turns` window by factor 0.1.

GroupChatSession

Orchestrates the full autonomous conversation loop:

1. Select next agent via `TurnScheduler`.
2. Trim history via `ContextWindow` (rolling or selective).
3. Call `agent.respond()`.
4. Persist message with HMAC author token via `ChatMessageService`.
5. Repeat `burst_size` times, reading fresh history between bursts.
6. Check `Group_chat.status` at each iteration — stop if `stopped`.

DM chat

`DMSession` (`src/dm_chat/session.py`) is a simplified single-agent path with no scheduler, no burst, no background task. Agent responds synchronously after each user message.

Evaluation Framework

The evaluation pipeline operates in two phases and measures four fidelity dimensions.

Four fidelity dimensions

Persona ID	Can judges correctly identify which anonymous author tag corresponds to which persona? Baseline: 12.5% (random over 8).
Individual	How faithfully did each agent represent its persona? Scored 1–5 per judge.
Group	Does the conversation exhibit realistic group dynamics (turn distribution, topic diversity)?
Behavioral	Does the conversation read as human-produced, or does it show LLM artifacts (excessive politeness, unnatural consensus)?

Phase 1 — Independent evaluation

All 20 judges evaluate independently. They are divided by role:

Role	Focus
style	Vocabulary, sentence structure, rhetorical habits
ideology	Political views, values, moral stances
behavioral	Behavioral fidelity, human-likeness
general	All traits combined

RAG pre-filtering. Before scoring, each judge queries ChromaDB to receive a shortlist of 20 candidate personas (from potentially 100+). Field-weighted retrieval boosts chunks relevant to the judge’s role. MMR re-ranking ensures diversity in the shortlist.

Hungarian assignment. Author $\leftrightarrow$ persona matching uses `scipy.optimize.linear_sum_assignment` to enforce a bijective assignment — each author maps to exactly one persona.

Statistical metrics

Persona identification.

$$\text{accuracy} = \frac{\text{correct guesses}}{\text{total guesses}} \quad (1)$$

$$H_0 : p = \frac{1}{8} = 0.125, \quad H_1 : p > 0.125 \quad (\text{one-sided binomial exact test}) \quad (2)$$

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (\text{Cohen’s Kappa, corrects for chance}) \quad (3)$$

Individual fidelity. Median + IQR over 20 Likert ratings per persona. Bootstrap 95% CI on the median (10 000 resamples). Judge-type agreement measured by MAD between median scores per type.

Group fidelity. Gini coefficient on turn distribution:

$$G = \frac{2 \sum_i i \cdot x_i}{n \sum_i x_i} - \frac{n+1}{n}$$

Target range for realistic group chats: $G \in [0.28, 0.42]$ (reference mean 0.33, std 0.05).

Batch evaluation. The batch pipeline (`src/eval/batch.py`) aggregates per-chat metrics across many sessions. CI method: bootstrap for $n < 20$ chats; CLT normal approximation ($CI = \bar{x} \pm 1.96 \cdot \sigma / \sqrt{n}$) for $n \geq 20$.

Phase 2 — Deliberation (design)

Phase 2 takes high-variance cases from Phase 1 into structured deliberation:

1. Identify cases where judge variance exceeds threshold.
2. Run 3–5 deliberation rounds. Judges explain evidence, revise ratings.
3. Track variance per round; convergence = $\text{var}_{\text{final}} < \text{var}_0$.
4. F-test between round 0 and final round variances.
5. Track confidence calibration: $r(\Delta\text{confidence}, \Delta\text{accuracy})$.

MCP Layer

The Model Context Protocol server (`src/mcp/mcp_server.py`) exposes the platform to any MCP-compatible LLM client via structured tool calls over stdio.

Architecture

```

LLM client (MCP host)
  | tool call (JSON over stdio)
  v
mcp_server.py [FastMCP]
  |-- tools/read.py      (Tier 1: free reads)
  \-- tools/write.py     (Tier 2: preview -> confirm)
      |
      v
      client.py [httpx -> FastAPI :8000]
          |
          v
          SQLite DB

```

Tool tiers

Tier 1 — Read (no confirmation required). `get_agents`, `get_agent_by_id`, `get_chat_messages`, `get_admin_overview`, `get_judge_result`, `get_judge_status`, and more. The `get_agents` tool strips the `summary` field (10k–50k tokens per agent) to prevent context overflow; full profiles retrieved by ID.

Tier 2 — Write (preview → confirm). Every write has two tools: `preview_*` shows the payload; `confirm_*` executes only after explicit user approval.

Preview / Confirm pair	Action
<code>create_full_chat</code>	Create DM or group chat
<code>create_message</code>	Post a message to a chat
<code>stop_chat</code>	Halt background agent loop
<code>start_judging</code>	Launch 20-judge evaluation (~\$0.40, ~9 min)
<code>create_topic</code>	Create a new topic

The LLM **never stops a chat autonomously** — only when the user explicitly writes “stop”.

Knowledge Graph

A pre-built knowledge graph of the entire codebase was committed at `graphify-out/graph.json` (6 920 nodes, 8 337 edges, generated via 88 LLM agent calls). The graph is organised into 20 community clusters:

Architecture & Core Concepts	Persona Agent Engine
Application Core & Auth	Persona Profile Extraction
Agent API & Repository	RAG Index Builder
User API & Service	UI & Admin Routes
Group Chat Runtime	UI Component Library
Judge Pipeline	Admin Dashboard UI
Evaluation Statistics Engine	... (20 clusters total)

God nodes (highest-degree abstractions): `UserService` (45 edges), `GroupChatService` (41), `TopicService` (39), `AgentService` (35).

Key architectural decisions captured as hyperedges: the five-mechanism persona context stack (structured profile + RAG + sliding window + reflection + perturbation); the two-phase judge pipeline; the anonymisation bridge between `Chat_messages` and `Agents`.

State-of-the-Art Positioning

Five papers ground the project methodology:

Wason et al. (2024) Identifies context management and response accuracy/dialogue success divergence as core unsolved challenges — directly motivating the sliding window, compression, and reflection mechanisms.

Salewski et al. (2023) Documents demographic bias in persona prompts. Warns that persona prompts specifying demographic characteristics induce stereotyped behavior — motivating the use of behavioral/ideological profiles rather than demographic labels.

Jiang et al. (2024) — **PersonaLLM** Validates that LLMs reliably encode Big Five trait profiles in self-reports and linguistic style. Critically: informing raters of AI authorship degrades personality attribution accuracy — motivating the **blind evaluation protocol** (anonymised author tags).

Zhou et al. (2025) Shows personal detail features are the single most critical driver of behavioral fidelity (removing them causes the largest accuracy drop). Motivates the structured profile schema.

Park et al. (2023) — Generative Agents Demonstrates memory, reflection, and planning for long-term coherent agents. Documents excessive agreeableness in group settings — motivating the perturbation mechanism.

Novel contribution. No prior work integrates all four fidelity dimensions (persona identification, individual, group, behavioral) in a unified experimental pipeline grounded in real-person text corpora. The project’s evaluation framework constitutes the novel scientific contribution.

Open Questions and Future Work

Deliberation (Phase 2) The phase 2 pipeline is designed (documented in `EXPERIMENTATION.MD` and `EVAL.md`) but not yet fully implemented. Implementing structured multi-round deliberation and measuring variance reduction would complete the evaluation pipeline.

Behavioral fidelity Unlike persona identification (clean baseline) and individual fidelity (Likert scores), behavioral fidelity — whether an agent reads as human-produced — lacks an operational definition. LIWC-based psycholinguistic profiling or human blind raters are the most viable approaches.

Optimal conversation length Too short (<20 turns) misses deep behavioral patterns; too long (>150 turns) causes drift. The empirically optimal range is not yet determined.

Candidate set size for identification The evaluation uses a closed 8-tag set (clean baseline, 12.5% random). An expanded-set design ($K > 8$ decoy personas) would be more ecologically valid but reduces statistical power. Three options are analyzed in `evaluation_design_open_questions.md`.

RAG recall for sparse personas Retrieval recall degrades for personas with <5k tokens of source material. No mitigation is implemented for sparse profiles.

MCP auth MCP read/write tools currently bypass FastAPI auth. The `Authorization` header injection needs to be added before production use.

Prefiltering experiments Varying candidate set size for judge shortlisting and measuring the effect on final accuracy would quantify the RAG retrieval quality vs. judge workload trade-off.

References

Jiang, H., Zhang, X., Cao, X., Breazeal, C., Roy, D., & Kabbara, J. (2024). **PersonaLLM: Investigating the Ability of Large Language Models to Express Personality Traits.** *Proceedings of ACL 2024*. arXiv:2305.02547.

Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). **Generative Agents: Interactive Simulacra of Human Behavior.** *Proceedings of UIST ’23*. ACM. <https://doi.org/10.1145/3586183.3606763>

Salewski, L., Alaniz, S., Rio-Torto, I., Schulz, E., & Akata, Z. (2023). **In-Context Impersonation Reveals Large Language Models’ Strengths and Biases.** *NeurIPS 2023*. arXiv:2305.14930.

Wason, R., Arora, P., Arora, D., Kaur, J., Singh, S. P., & Hoda, M. N. (2024). **Appraising Success of LLM-based Dialogue Agents.** *INDIACom 2024*. IEEE.

Zhou, J., Jiang, B., Gong, R., & Jiang, H. (2025). **Evaluating LLM-based Role-playing Agent Authenticity via Event-driven Response Simulation.** *ISAICS 2025*. IEEE. DOI: 10.1109/ISAICS66888.2025.11350200.